LAPORAN PRAKTIKUM

Pengembangan Aplikasi Web II

Disusun oleh:

Nama : R. A. Rafa Lutfyah
NPM : 2428240125
Kelas : SI5C

PROGRAM STUDI SISTEM INFORMASI
UNIVERSITAS MULTI DATA PALEMBANG
TAHUN AKADEMIK 2026/2027

Identitas Praktikum

Nama Mahasiswa	R. A. Rafa Lutfyah
NPM	2428240125
Kelas	SI5C
Pertemuan Ke-	3 dan 4 (Tugas Mandiri)
Topik / Judul Praktikum	Membangun RESTful API Murni dengan Express.js Topik 14 – Maskapai : Jadwal Penerbangan
Tanggal Pelaksanaan	24 September 2026
Dosen Pengampu	Nur Rachmat M.Kom

I. Tujuan Praktikum

1. Membangun satu RESTful API murni dengan Express.js untuk entitas Jadwal Penerbangan (Topik 14 - Maskapai), tanpa tampilan (view/HTML) apa pun, sesuai topik yang ditetapkan berdasarkan nomor absen.

2. Merancang tabel endpoint RESTful (GET, POST, PUT, DELETE, dan filter dengan query string) untuk resource /flights sebelum menulis kode.

3. Mengimplementasikan operasi CRUD lengkap menggunakan data sementara (array) di memori, mengikuti format response JSON standar: data polos untuk response GET (objek untuk satu data, array untuk banyak data), dan struktur { status, message, data } untuk response POST, PUT, DELETE, serta error (400/404).

4. Menguji setiap endpoint yang telah diimplementasikan menggunakan Postman, termasuk minimal dua skenario gagal (400 dan 404), serta mendokumentasikan hasilnya.

5. Mem-push project ke repository GitHub publik (tugas1-restful-2428240125) dengan commit bertahap, dan men-deploy project ke Vercel.

II. Dasar Teori

Tugas ini merupakan penerapan langsung dari materi RESTful API dengan Express.js pada entitas yang dipilih sesuai pembagian topik pada Tugas 1 Mandiri, yaitu Topik 14 - Maskapai: Jadwal Penerbangan dengan endpoint dasar /flights (Rachmat, n.d.).

Sebelum menulis kode, dilakukan perancangan terlebih dahulu daftar endpoint yang dibutuhkan, mencakup method HTTP (GET, POST, PUT, DELETE), path endpoint, deskripsi, dan kode status yang diharapkan. Rancangan ini menjadi acuan saat implementasi (Rachmat, n.d.).

Konvensi penamaan resource pada RESTful API menggunakan huruf kecil dan bentuk jamak tanpa kata kerja, misalnya /flights (bukan /getFlights atau /Flight), dan route

parameter hanya dipakai untuk :id, sedangkan filter data memakai query string yang dibaca melalui req.query (Rachmat, n.d.).

Response GET dikirim polos tanpa bungkus status/message (objek untuk satu data, array untuk banyak data, termasuk hasil filter yang boleh berupa array kosong), sedangkan response POST, PUT, DELETE, dan error dibungkus dalam struktur { status, message, data } agar konsumen API dapat membedakan hasil operasi secara konsisten (Rachmat, n.d.).

Data yang diterima dari client perlu divalidasi di sisi server. Pada entitas jadwal penerbangan, field kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat wajib diisi; jika salah satu kosong, server membalas status 400 (Bad Request).

Kode status yang konsisten menggunakan 2xx untuk keberhasilan (200 untuk GET/PUT/DELETE, 201 untuk POST) dan 4xx untuk kegagalan pada sisi client (400 untuk data tidak lengkap, 404 untuk data/endpoint tidak ditemukan) (MDN Web Docs, n.d.).

III. Alat dan Bahan

No.	Nama Alat / Software	Versi / Keterangan
1	Node.js	Runtime JavaScript, versi LTS (Express 5 mensyaratkan Node.js 18 ke atas)
2	Visual Studio Code	Editor kode dan terminal terintegrasi
3	npm	Package manager, terpasang bersama Node.js
4	Express.js	Versi 5.2.1 (dependencies)
5	nodemon	Versi 3.1.14 (devDependencies)
6	Postman	Aplikasi untuk menguji endpoint (GET, POST, PUT, DELETE) beserta query string filter
7	Git & GitHub	Version control dan hosting repository publik tugas1-restful-2428240125
8	Vercel	Platform deployment agar API dapat diakses secara publik

IV. Langkah Kerja

1. Memilih entitas dari pembagian topik Tugas 1 sesuai nomor absen, yaitu Topik 14 - Jadwal Penerbangan (Maskapai), dengan atribut: kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat.

2. Merancang tabel endpoint RESTful untuk resource /flights sebelum menulis kode (lihat Tabel Rancangan Endpoint pada Bagian V).

3. Membuat folder project tugas1-restful-2428240125, menjalankan npm init -y, mengatur "main": "app.js" beserta script "start" dan "dev", lalu memasang Express.js sebagai dependency dan nodemon sebagai devDependency.

```
npm init -y
npm install express
npm install --save-dev nodemon
```

4. Membuat berkas app.js, mendaftarkan middleware express.json(), dan menyiapkan data sementara berupa array flights (3 data awal) beserta variabel nextId.

```
const express = require("express");
const app = express();
const PORT = process.env.PORT || 3000;
app.use(express.json());

let flights = [
  { id: 1, kodePenerbangan: "XY-512", maskapai: "Nusantara Air",
    asal: "CGK", tujuan: "DPS", jadwalBerangkat: "2026-10-20T07:30:00+07:00" },
  { id: 2, kodePenerbangan: "QG-880", maskapai: "Citra Wings",
    asal: "PLM", tujuan: "CGK", jadwalBerangkat: "2026-10-21T09:15:00+07:00" },
  { id: 3, kodePenerbangan: "XY-330", maskapai: "Nusantara Air",
    asal: "CGK", tujuan: "SUB", jadwalBerangkat: "2026-10-22T13:00:00+07:00" },
];
let nextId = 4;
```

5. Membuat fungsi validasi() untuk memeriksa field wajib (kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat). Fungsi ini dipakai bersama oleh route POST dan PUT.

```
function validasi(body) {
  const { kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat } = body || {};
  if (!kodePenerbangan || !maskapai || !asal || !tujuan || !jadwalBerangkat) {
    return "kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat wajib diisi";
  }
  return null;
}
```

6. Membuat route GET / sebagai info API (nama, NIM, kelas, topik, dan daftar endpoint), route GET /flights untuk menampilkan seluruh data sekaligus mendukung filter ?asal= melalui req.query, serta GET /flights/:id yang membalas 404 (dibungkus { status, message, data }) jika id tidak ditemukan.

```
// GET /flights -> semua data, atau hasil filter ?asal=...
app.get("/flights", (req, res) => {
  const { asal } = req.query;
  if (asal) {
    const hasil = flights.filter((f) => f.asal === asal);
    return res.json(hasil);
  }
  res.json(flights);
});

// GET /flights/:id -> satu data berdasarkan id
app.get("/flights/:id", (req, res) => {
  const id = parseInt(req.params.id);
  const data = flights.find((f) => f.id === id);
  if (!data) {
    return res.status(404).json({
```

```
      status: "error",
      message: `Data dengan id ${id} tidak ditemukan`,
      data: null,
    });
  }
  res.json(data);
});
```

7. Membuat route POST /flights untuk menambahkan data baru. Jika validasi gagal, server membalas status 400; jika berhasil, id dibuat otomatis (nextId++) dan server membalas status 201.

```
app.post("/flights", (req, res) => {
  const pesan = validasi(req.body);
  if (pesan) {
    return res.status(400).json({ status: "error", message: pesan, data: null });
  }
  const { kodePenerbangan, maskapai, asal, tujuan, jadwalBerangkat } = req.body;
  const baru = { id: nextId++, kodePenerbangan, maskapai, asal, tujuan,
jadwalBerangkat };
  flights.push(baru);
  res.status(201).json({
    status: "success",
    message: "Data jadwal penerbangan berhasil ditambahkan",
    data: baru,
  });
});
```

8. Membuat route PUT /flights/:id (penggantian penuh; membalas 404 jika data tidak ada dan 400 jika validasi gagal) dan DELETE /flights/:id (membalas 404 jika data tidak ada, dan 200 dengan data: null jika berhasil), memakai pola findIndex().

9. Menambahkan middleware catch-all di akhir file untuk menangani route yang tidak terdaftar, membalas status 404 dengan pesan "Endpoint tidak ditemukan", serta menyesuaikan app.listen dan module.exports = app agar dapat berjalan di Vercel.

10. Menjalankan server dengan npm run dev, lalu menguji seluruh endpoint melalui Postman, mencakup skenario berhasil maupun gagal (400 dan 404). Selanjutnya melakukan commit bertahap ke Git (minimal 5 kali), push ke GitHub, dan men-deploy ke Vercel.

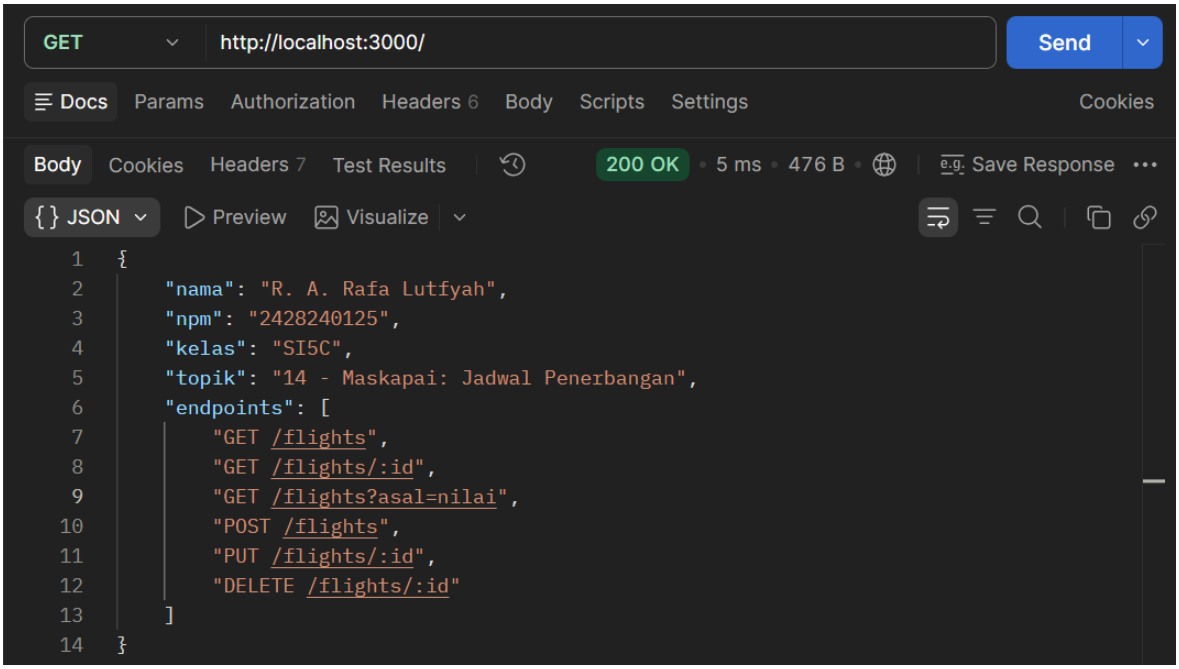
V. Hasil dan Pembahasan

Tabel rancangan endpoint (langkah 2):

No.	Method	Endpoint	Deskripsi	Status
1	GET	/flights	Menampilkan seluruh data jadwal penerbangan	200
2	GET	/flights/:id	Menampilkan satu data jadwal penerbangan berdasarkan id	200 / 404
3	POST	/flights	Menambah data jadwal penerbangan baru	201 / 400

No.	Method	Endpoint	Deskripsi	Status
4	PUT	/flights/:id	Mengubah seluruh data jadwal penerbangan (penggantian penuh)	200 / 400 / 404
5	DELETE	/flights/:id	Menghapus data jadwal penerbangan	200 / 404
6	GET	/flights?asal=nilai	Filter data jadwal penerbangan berdasarkan kode bandara asal (req.query)	200

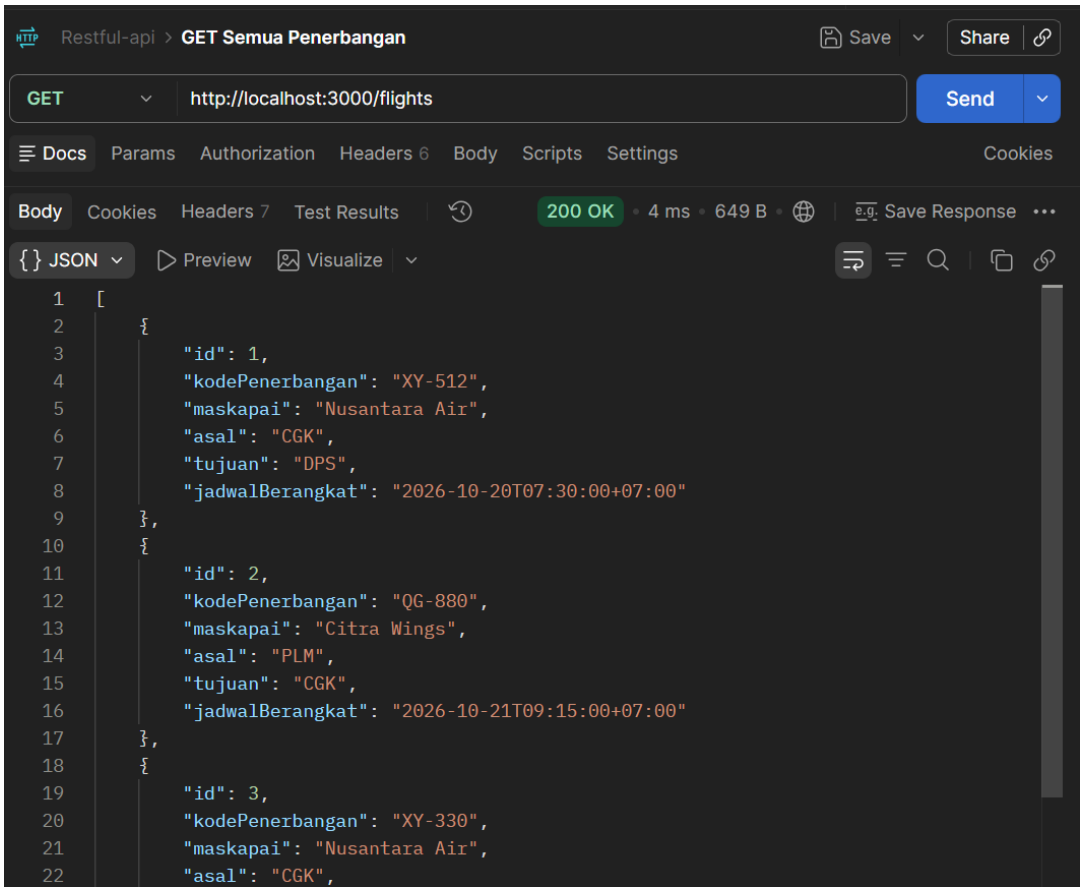
Verifikasi endpoint GET / (info API - Soal 1 poin 5):



Gambar 1. Response GET / menampilkan nama, npm, kelas, topik, dan daftar endpoint (200 OK)

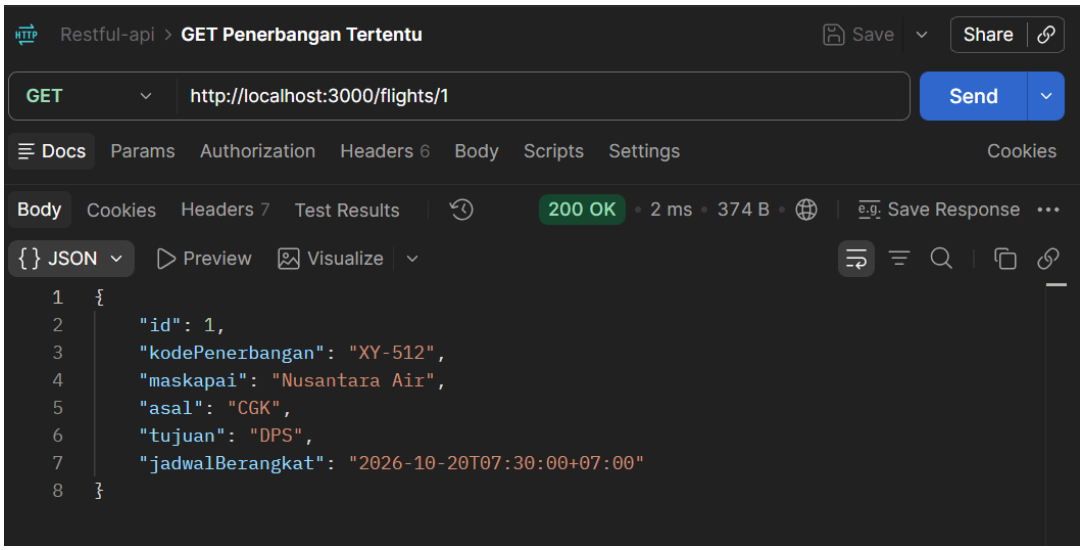
Tangkapan layar pengujian (Postman), sesuai daftar pengujian Bagian E.1:

Pengujian 1: GET /flights (harapan: 200)



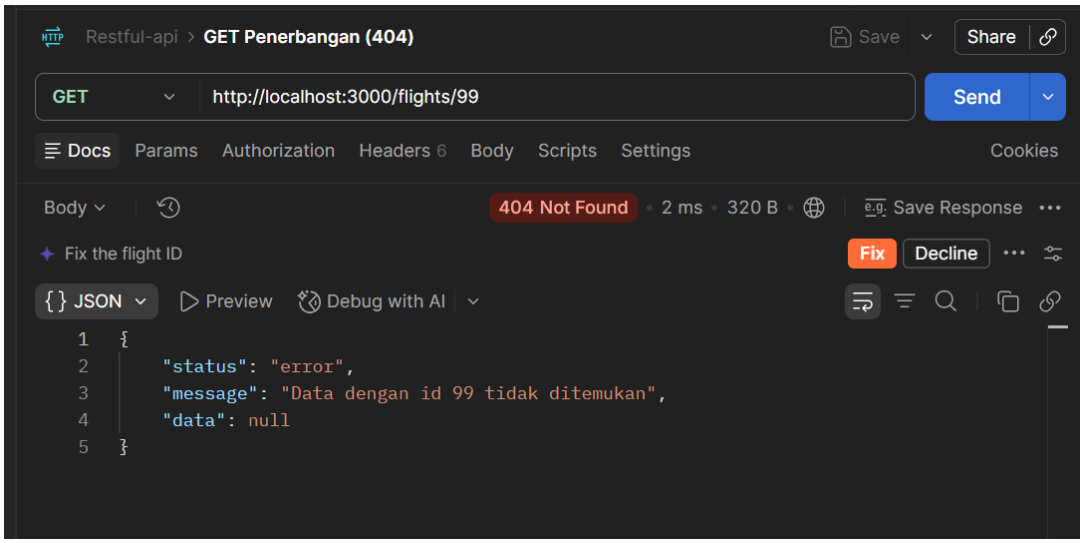
Gambar 2. GET /flights - menampilkan seluruh data (200 OK)

Pengujian 2: GET /flights/1 (harapan: 200)



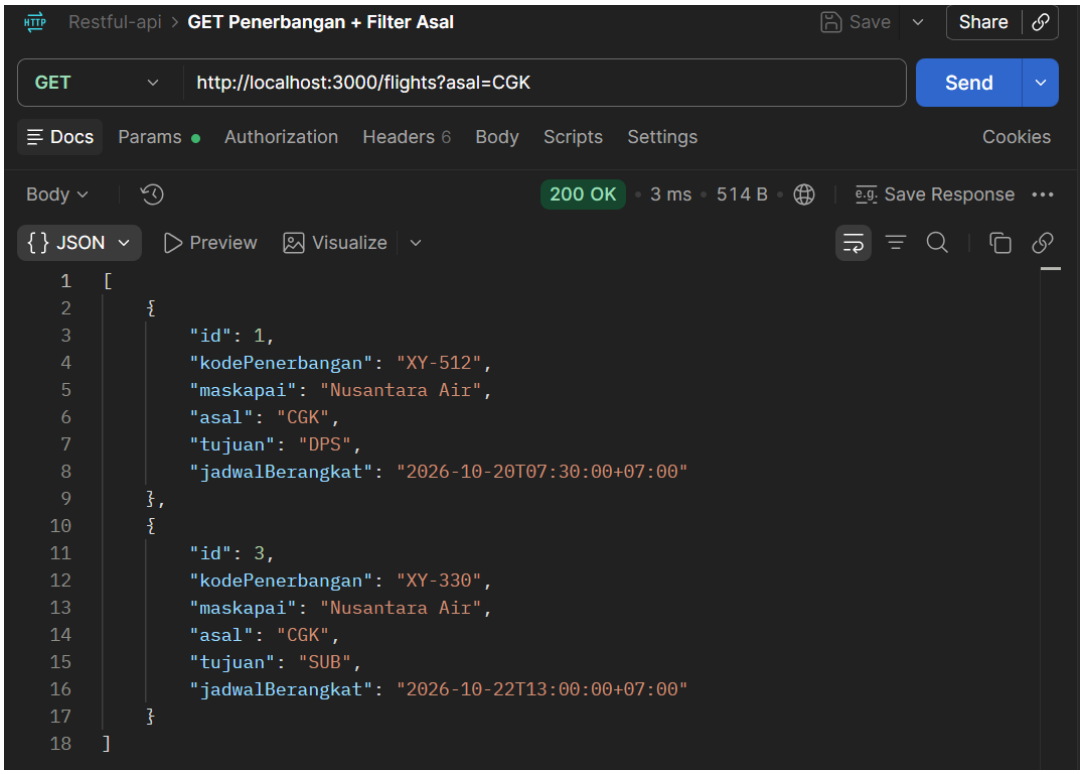
Gambar 3. GET /flights/1 - menampilkan satu data (200 OK)

Pengujian 3: GET /flights/99 (harapan: 404)



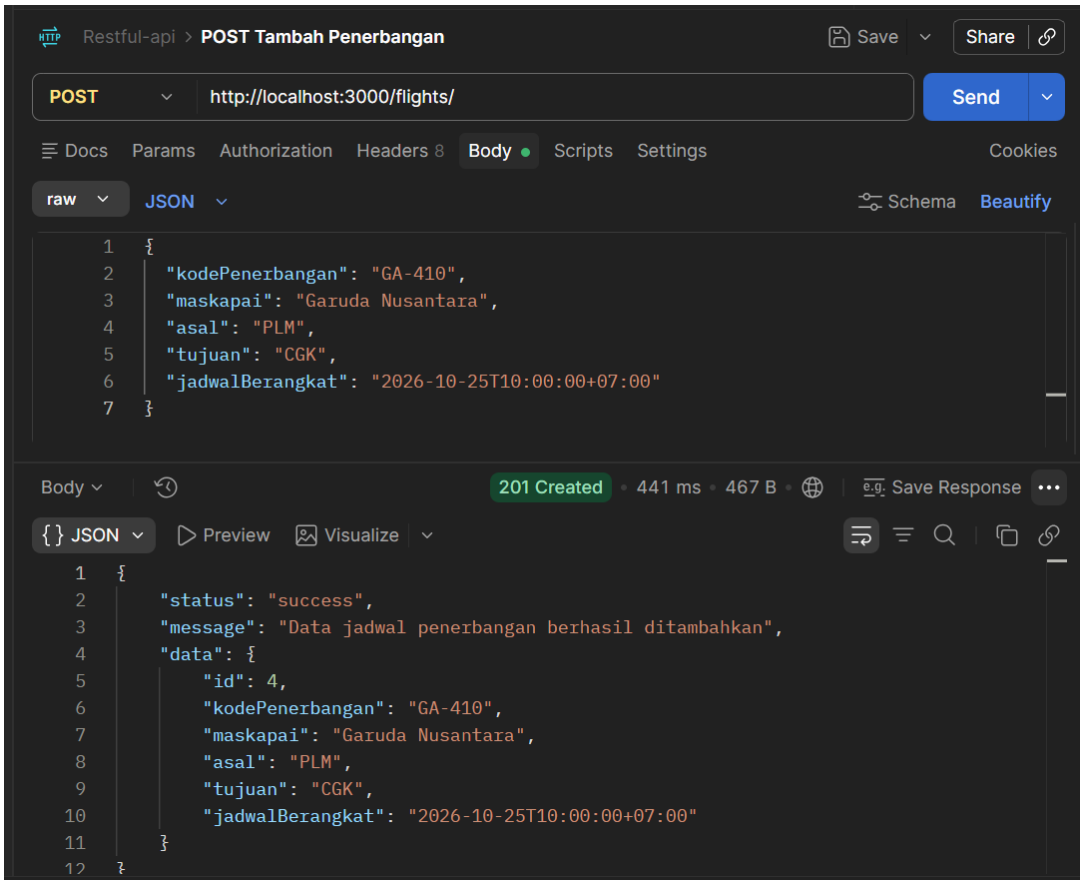
Gambar 4. GET /flights/99 - id tidak ditemukan (404 Not Found)

Pengujian 4: GET /flights?asal=CGK (harapan: 200)



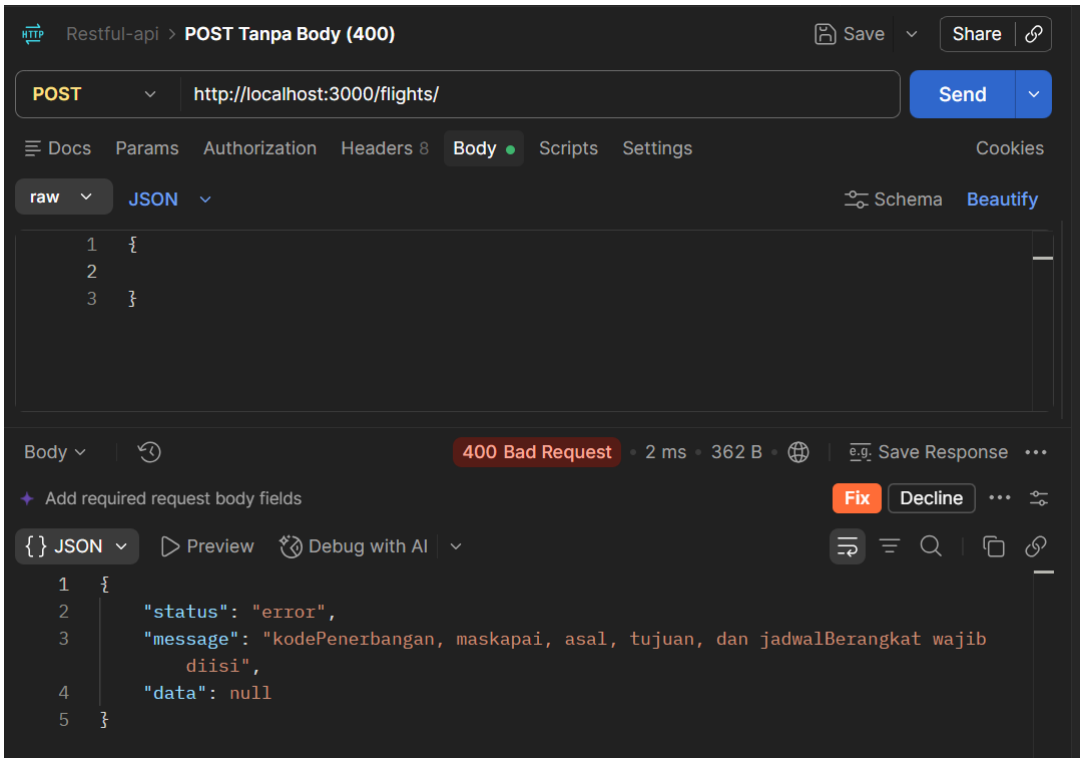
Gambar 5. GET /flights?asal=CGK - filter query string (200 OK)

Pengujian 5: POST /flights - body lengkap (harapan: 201)



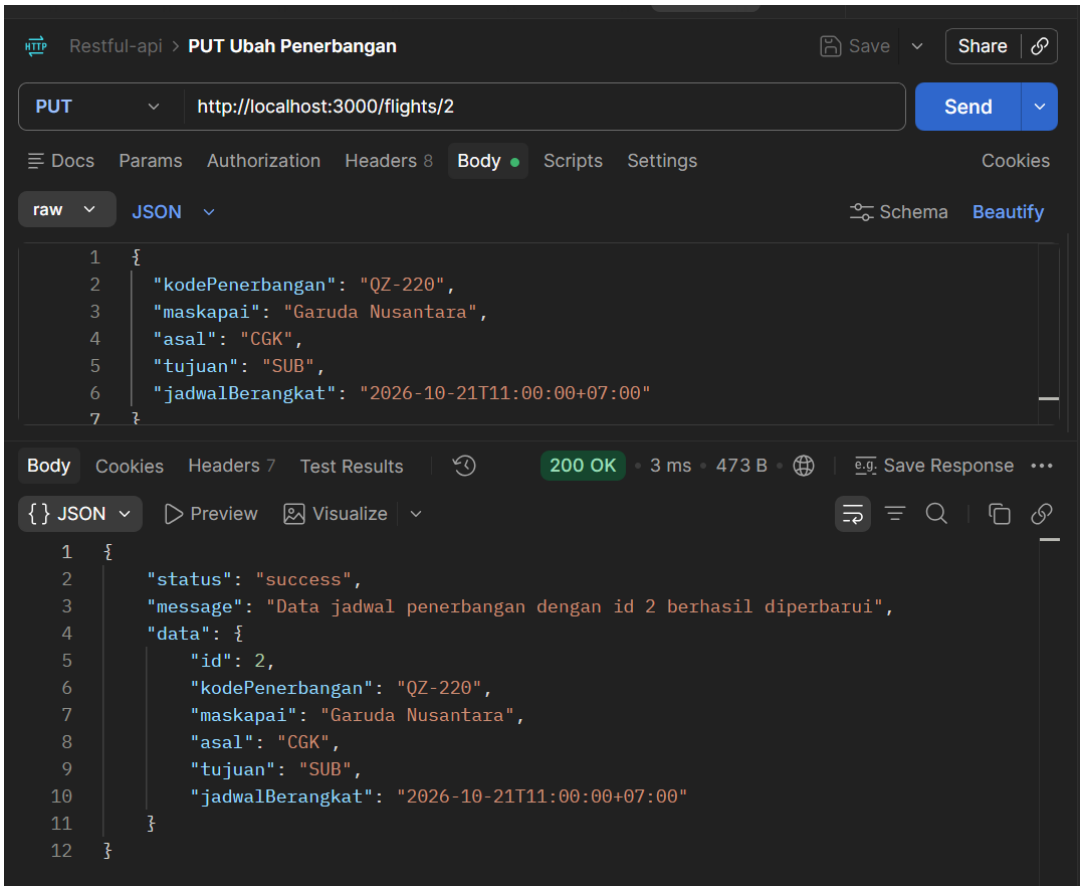
Gambar 6. POST /flights - body lengkap (201 Created)

Pengujian 6: POST /flights - field wajib kosong ({}) (harapan: 400)



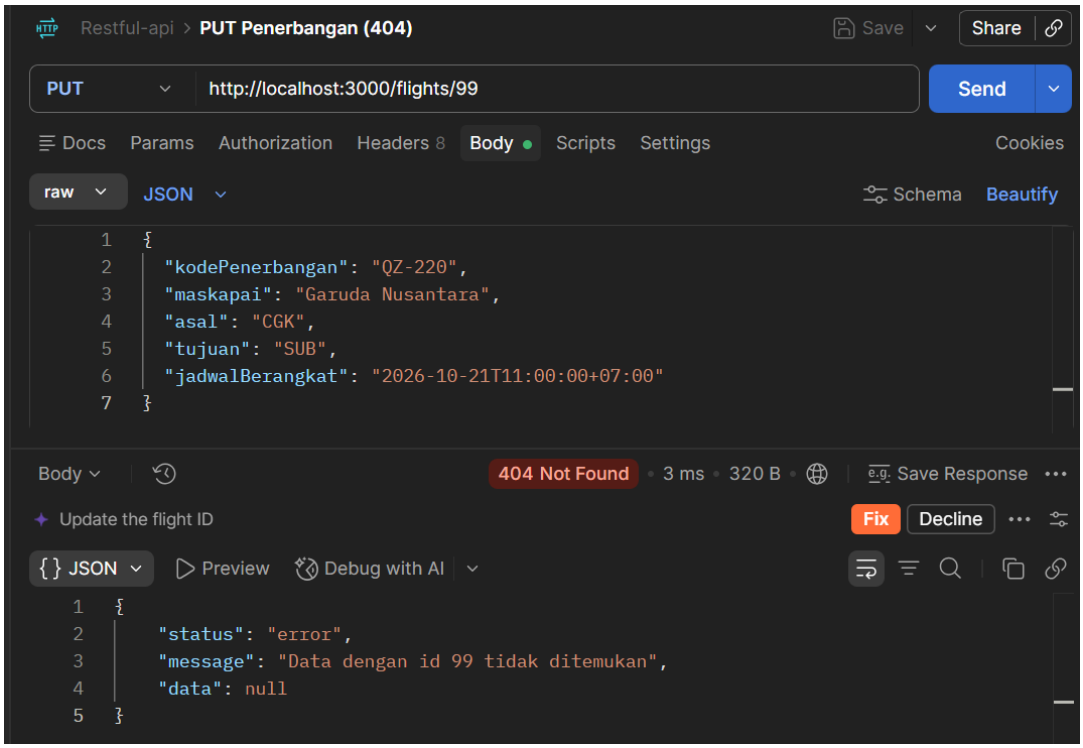
Gambar 7. POST /flights dengan body kosong - validasi gagal (400 Bad Request)

Pengujian 7: PUT /flights/2 - body lengkap (harapan: 200)



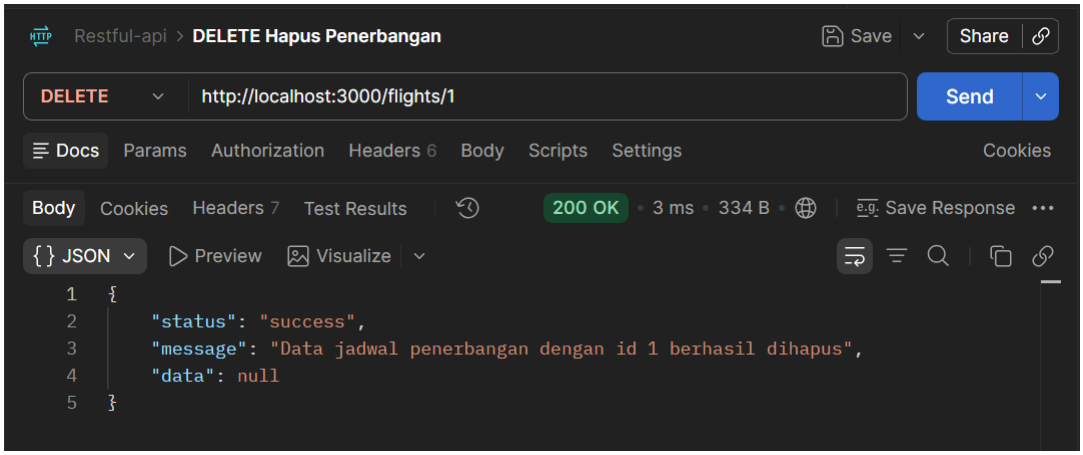
Gambar 8. PUT /flights/2 - body lengkap (200 OK)

Pengujian 8: PUT /flights/99 - body lengkap (harapan: 404)



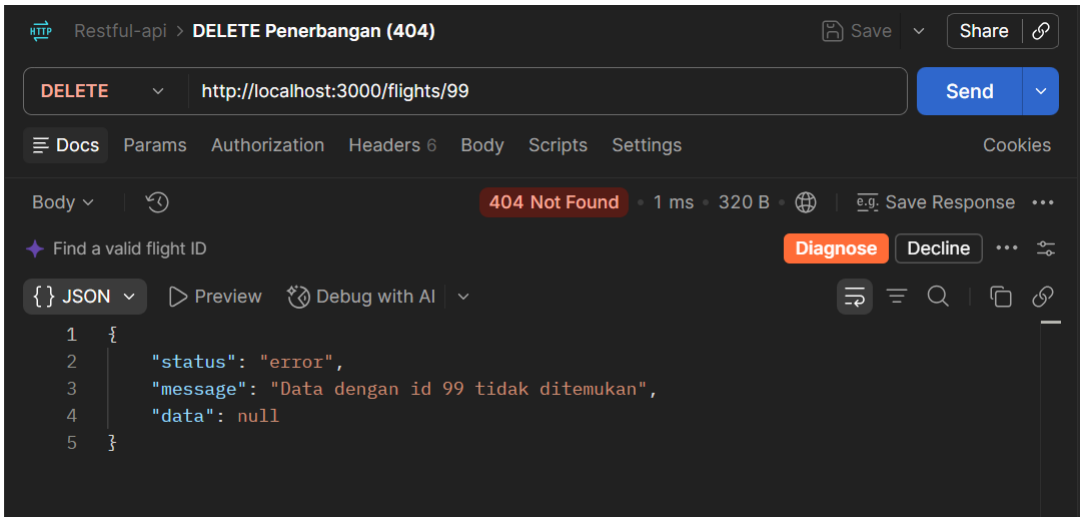
Gambar 9. PUT /flights/99 - id tidak ditemukan (404 Not Found)

Pengujian 9: DELETE /flights/1 (harapan: 200)



Gambar 10. DELETE /flights/1 (200 OK)

Pengujian 10: DELETE /flights/99 (harapan: 404)



Gambar 11. DELETE /flights/99 (404 Not Found)

Pembahasan:

GET /flights mengembalikan array seluruh data jadwal penerbangan secara polos (tanpa status/message), sesuai ketentuan format response GET pada Tugas 1. GET /flights/1 mengembalikan objek data secara polos untuk id yang ditemukan (200), sedangkan GET /flights/99 mengembalikan format { status: "error", message, data: null } dengan status 404 untuk id yang tidak terdaftar. Method GET dipilih karena hanya membaca data dan tidak mengubah array.

Query string ?asal=CGK pada GET /flights berhasil memfilter data berdasarkan kode bandara asal (req.query) dan tetap mengembalikan status 200 sesuai ketentuan B.1, walaupun seandainya hasilnya array kosong ([]).

POST dipilih untuk menambah data baru karena membuat resource baru pada koleksi /flights. Dengan body lengkap, server membalas status 201 (Created) beserta objek data baru yang dibungkus { status, message, data }, dengan id dibuat otomatis oleh server. Ketika body dikirim kosong ({}), server membalas status 400 (Bad Request) dengan pesan "kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat wajib diisi".

PUT dipilih untuk penggantian penuh data pada id tertentu: seluruh field wajib dikirim ulang dan id tidak berubah. Jika berhasil, server membalas 200 beserta data terbaru; jika id tidak ada dibalas 404, dan jika field wajib tidak lengkap dibalas 400 - keduanya memakai struktur { status, message, data }.

DELETE dipilih untuk menghapus data pada id tertentu. Jika berhasil, server membalas 200 dengan data: null dan pesan yang menyebut id yang dihapus; jika id tidak ditemukan dibalas 404. Permintaan ke route yang tidak terdaftar sama sekali ditangani middleware catch-all dan tetap membalas JSON dengan status 404 dan pesan "Endpoint tidak ditemukan", bukan halaman error bawaan Express.

Ringkasan hasil pengujian:

No.	Method	Endpoint	Data request	Status diharapkan	Status hasil	Keterangan
1	GET	/flights	-	200	200 OK	Sesuai; array seluruh data (3 data awal)
2	GET	/flights/1	-	200	200 OK	Sesuai; objek data id 1 (XY-512)
3	GET	/flights/99	-	404	404 Not Found	Sesuai; id tidak ditemukan, data: null
4	GET	/flights?asal=...	-	200	200 OK	Sesuai; 2 data berasal CGK (id 1 dan 3)
5	POST	/flights	body lengkap	201	201 Created	Sesuai; data baru dengan id 4 (GA-410)
6	POST	/flights	field wajib kosong ({})	400	400 Bad Request	Sesuai; field wajib tidak diisi
7	PUT	/flights/2	body lengkap	200	200 OK	Sesuai; data id 2 berhasil diperbarui
8	PUT	/flights/99	body lengkap	404	404 Not Found	Sesuai; id tidak ditemukan, data: null
9	DELETE	/flights/1	-	200	200 OK	Sesuai; data id 1 dihapus, data: null
10	DELETE	/flights/99	-	404	404 Not Found	Sesuai; id tidak ditemukan, data: null

VI. Kendala dan Solusi

No.	Kendala	Solusi
1	Bingung membedakan dua format response: GET harus mengirim data polos, sedangkan POST/PUT/DELETE/error harus dibungkus { status, message, data }.	Menuliskan res.json(flights) apa adanya untuk semua route GET, dan konsisten membungkus res.status(...).json({ status, message, data }) hanya pada POST, PUT, DELETE, dan setiap respons error (400/404).
2	Nilai jadwalBerangkat yang ditulis tanpa tanda kutip (bukan string) ditolak oleh JSON.parse Postman sebelum request terkirim.	Menuliskan jadwalBerangkat sebagai string berformat ISO 8601 dengan tanda kutip, misalnya "jadwalBerangkat": "2026-10-20T07:30:00+07:00".

No.	Kendala	Solusi
3	Data yang disimpan di memori (array flights) dapat kembali ke data awal setelah beberapa saat ketika API sudah di-deploy ke Vercel.	Memahami bahwa hal ini wajar pada lingkungan serverless (setiap invocation dapat memulai instance baru) dan tidak mengurangi nilai, selama setiap request tunggal tetap mendapat response yang benar sesuai skenario pengujian.

VII. Kesimpulan

1. Entitas Jadwal Penerbangan (Maskapai, Topik 14) berhasil diimplementasikan sebagai RESTful API murni dengan resource /flights dan atribut kodePenerbangan, maskapai, asal, tujuan, dan jadwalBerangkat (tujuan 1).

2. Tabel rancangan endpoint untuk keenam skenario (GET semua data, GET satu data, POST, PUT, DELETE, dan filter) telah dibuat sebelum implementasi (tujuan 2).

3. Seluruh endpoint pada rancangan berhasil diimplementasikan menggunakan Express.js dengan data sementara di memori, serta mengikuti format response JSON standar: data polos untuk GET, dan { status, message, data } untuk POST/PUT/DELETE/error (tujuan 3).

4. Pengujian dengan Postman menunjukkan seluruh skenario pada tabel pengujian memberikan response dan kode status yang sesuai rancangan, termasuk skenario gagal (400 dan 404) (tujuan 4).

5. Project telah di-push ke repository GitHub publik dan di-deploy ke Vercel sesuai ketentuan Soal 5 (lihat tautan pada Lampiran) (tujuan 5).

Dengan demikian, tugas ini telah memenuhi kriteria kelengkapan endpoint CRUD beserta filter query string sesuai ketentuan pada Bagian B soal Tugas 1.

Daftar Pustaka

Express. (n.d.). Routing. Diakses 28 September 2026, dari https://expressjs.com/en/guide/routing.html

MDN Web Docs. (n.d.). HTTP response status codes. Mozilla. Diakses 28 September 2026, dari https://developer.mozilla.org/en-US/docs/Web/HTTP/Status

Rachmat, N. (n.d.). Membangun RESTful API dengan Express.js [Materi perkuliahan, Pertemuan 3-4, termasuk Tugas 1 Mandiri]. Pengembangan Aplikasi Web II, Program Studi Sistem Informasi, Universitas Multi Data Palembang.

Lampiran

Tautan Repository GitHub:

https://github.com/raden2428240125/tugas1-restful-2428240145

Tautan Deployment (Vercel):

https://tugas1-restful-2428240145.vercel.app/